



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

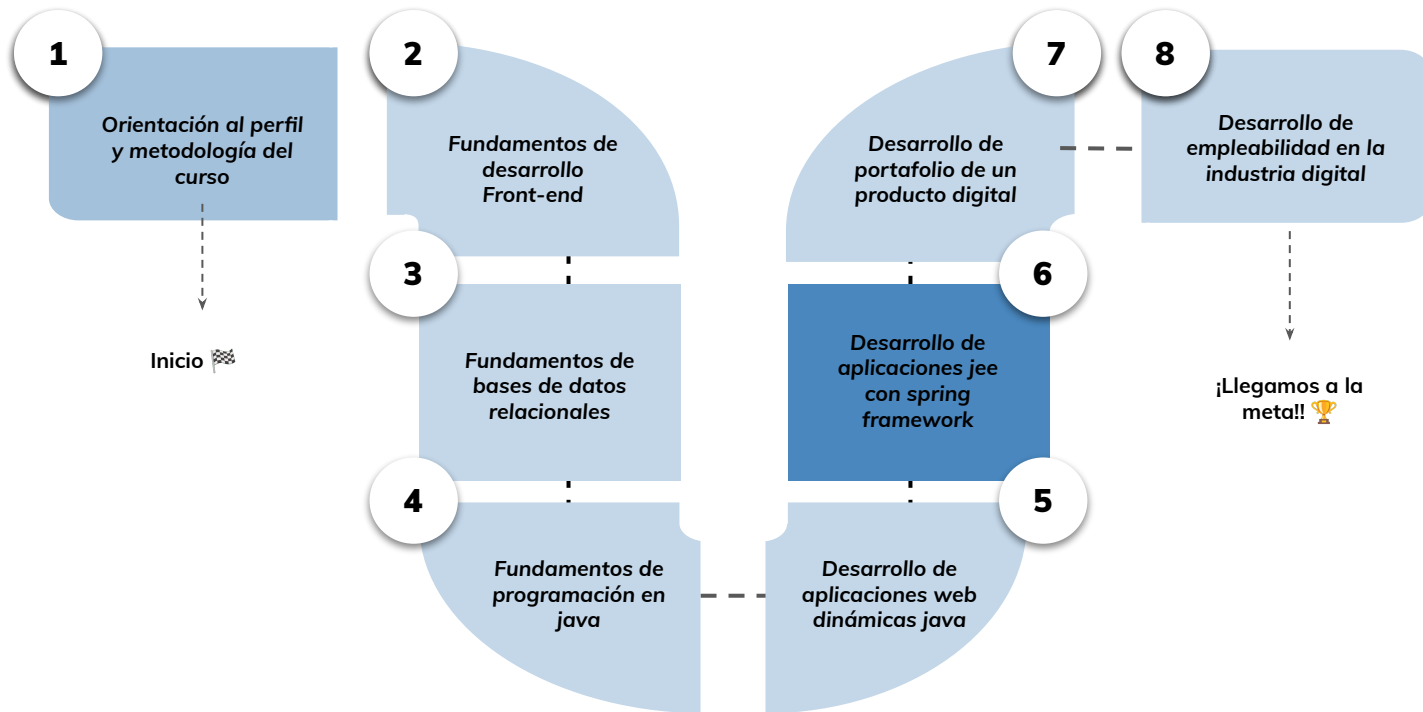


› Acceso a datos en spring framework - Parte II

Aprendizaje Esperado: Implementar capa de acceso a datos en una aplicación web utilizando spring framework para dar solución a un problema.

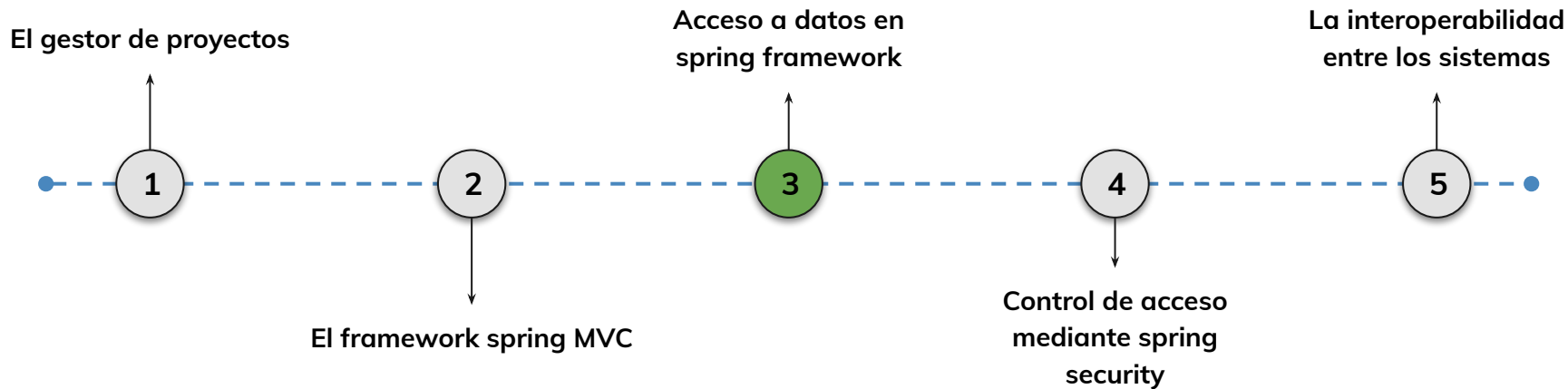
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Implementar capa de acceso a datos en una aplicación web utilizando Spring Framework

Aprenderemos a utilizar Spring Data JPA para definir repositorios, realizar asociaciones entre entidades y manejar transacciones de forma segura y eficiente.

Repositorios JPA

Creación de interfaces
CrudRepository y
JpaRepository

Consultas automáticas y
personalizadas con @Query

Relaciones y
transacciones

Mapeo de relaciones
@OneToOne y @OneToMany

Uso de @Transactional,
propagación y aislamiento



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender el rol de los repositorios en JPA
- Usar interfaces como CrudRepository o JpaRepository
- Implementar consultas personalizadas con @Query
- Mapear relaciones @OneToOne y @OneToMany
- Usar transacciones con @Transactional
- Configurar propagación y aislamiento transaccional

Repaso clase anterior

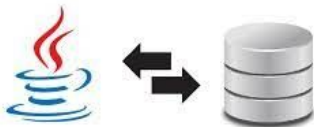
¿Quedó alguna duda?

En la clase anterior trabajamos :

- Configuración del DataSource en Spring
- Uso de JdbcTemplate para operaciones CRUD
- Implementación de DAO con consultas parametrizadas
- Mapeo de resultados a objetos con RowMapper

➤ Clases de Repositorio

Clases de Repositorio



JAVA PERSISTENCE API

En el contexto de Spring y Java Persistence API (JPA), las clases de repositorio desempeñan un papel fundamental en la capa de persistencia de una aplicación. Estas clases facilitan la interacción entre la lógica de negocio y la capa de acceso a datos, proporcionando métodos convenientes para realizar operaciones CRUD en las entidades persistentes.

Clases de Repositorio

Las Clases de Repositorio en Spring Data JPA son interfaces o clases que extienden las interfaces proporcionadas por Spring Data y que heredan de **JpaRepository** o **CrudRepository**. Su principal objetivo es proporcionar una abstracción sobre las operaciones básicas de persistencia, permitiendo a los desarrolladores interactuar con la base de datos de manera más sencilla y orientada a objetos.

Clases de Repositorio

Para crear una interfaz repositorio, primero debemos anotarla con `@Repository`. Para poder realizar esto es necesario tener la dependencia de **Spring Data JPA** en el archivo **pom.xml** de nuestra aplicación.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Clases de Repositorio

En este ejemplo, la interfaz **UsuarioRepository** extiende de **CrudRepository** y trabaja con la entidad **Usuario** cuya clave primaria es de tipo **String**.

```
8 @Repository
9 public interface UsuarioRepository extends CrudRepository<Usuario, String> {
10
11 }
```

Clases de Repositorio

Características Principales:

- **Generación de Consultas Automáticas** : Spring Data JPA realiza la generación automática de consultas basadas en el nombre del método. Por ejemplo, al definir un método `findById`, Spring Data JPA generará automáticamente la consulta SQL para buscar un usuario por su id.
- **Soporte para Métodos de Convención de Nombres** : Los métodos en las interfaces de repositorio pueden seguir una convención de nombres, y Spring Data JPA los interpretará automáticamente para crear consultas. Esto incluye métodos como `findBy`, `findAllBy`, `deleteBy`, etc.
- **Consultas Personalizadas** : Además de las consultas automáticas, los repositorios JPA permiten la definición de consultas personalizadas mediante el uso de anotaciones como `@Query`. Esto ofrece flexibilidad cuando se necesitan consultas más complejas.

Acceso a datos con JPA

Ventajas y Mejores Prácticas:

Abstracción y Simplicidad : Las clases de repositorio abstraen gran parte de la complejidad del acceso a datos, permitiendo a los desarrolladores centrarse en la lógica de negocio.

Reducción de Código Boilerplate : Spring Data JPA reduce la necesidad de escribir código repetitivo y de bajo nivel para realizar operaciones CRUD.

Facilita las Pruebas Unitarias : Al proporcionar una interfaz, las clases de repositorio facilitan la creación de pruebas unitarias al permitir la sustitución de implementaciones para realizar pruebas de manera más efectiva.

Integración con Spring Boot : En aplicaciones basadas en Spring Boot, la configuración y la inicialización de repositorios son manejadas automáticamente, simplificando aún más el desarrollo.

➤ Recuperar, actualizar,
eliminar un objeto en JPA

Recuperar, actualizar, eliminar un objeto en JPA

Recuperar, actualizar y eliminar son operaciones fundamentales que permiten interactuar con la capa de persistencia de manera efectiva. A continuación, analicemos en detalle cómo se llevan a cabo estas operaciones en JPA con Spring, abordando aspectos teóricos, mejores prácticas y consideraciones relevantes.

Recuperar, actualizar, eliminar un objeto en JPA

CrudRepository es una interfaz genérica que recibe dos tipos. El primero es la clase que esta interfaz maneja y el segundo es el tipo de dato del ID de la entidad.

Simplemente con crear la **interfaz UsuarioRepository** y hacerla extender de **CrudRepository** ya podemos hacer uso de diversos métodos que se encuentran previamente declarados de manera automática (sin necesidad de escribir más código).

```
8 @Repository
9 public interface UsuarioRepository extends CrudRepository<Usuario, String> {
10
11 }
```

Recuperar, actualizar, eliminar un objeto en JPA

Los métodos que nos provee CrudRepository son:

- **save**: guarda una entidad.
- **saveAll**: guarda las entidades de una lista iterable.
- **findById**: busca por el identificador.
- **existsById**: verifica si existe un identificador.
- **findAll**: devuelve todos los elementos para la entidad.

Recuperar, actualizar, eliminar un objeto en JPA

- **findAllById** : busca todos los elementos que tengan el identificador.
- **count** : devuelve el total de registros de la entidad.
- **deleteById** : elimina un registro según el identificador.
- **delete** : elimina la entidad.
- **deleteAllById** : elimina todos los elementos que correspondan con el identificador.
- **deleteAll (Iterable)** : elimina todos los elementos que se reciban en el parámetro.
- **deleteAll** : elimina todos los elementos.

Recuperar, actualizar, eliminar un objeto en JPA

```
@Repository
public interface UsuarioRepository extends CrudRepository<Usuario, String> {
    // Spring Data JPA proporciona métodos CRUD estándar
    // sin necesidad de escribir implementaciones.

    // Métodos para Leer (Retrieve) por ID:
    public Optional<Usuario> findById(Long id);
    public List<Usuario> findAll();

    // Métodos para Crear (Save):
    public Usuario save(Usuario usuario); // También se utiliza para actualizar

    // Métodos para Eliminar (Delete):
    public void deleteById(Long id);
    public void delete(Usuario usuario);
    public void deleteAll();
    public void deleteAll(Iterable<? extends Usuario> entities);
}
```

Recuperar, actualizar, eliminar un objeto en JPA

Si los métodos anteriores no son funcionales para realizar consultas específicas, podemos declarar métodos de búsqueda propios en la interfaz. Utilizando la convención de nomenclatura **jpql** podremos realizar el siguiente tipo de consultas:

```
13 @Repository
14 public interface UsuarioRepository extends CrudRepository<Usuario, String> {
15
16     @Query("SELECT u FROM Usuario u WHERE u.name = :name")
17     public Usuario buscarPorNombre(@Param("name") String name);
18 }
```

Recuperar, actualizar, eliminar un objeto en JPA

En esta query personalizada, podemos realizar una búsqueda a través de los parámetros señalados. La anotación `@Query` es necesaria para que Spring sepa que tiene que evaluarlo como una consulta a base de datos.

El parámetro `":nombre"` hace referencia al señalado en la anotación `@Param("nombre")`, seguido por el tipo de dato a evaluar.

```
13 @Repository
14 public interface UsuarioRepository extends CrudRepository<Usuario, String> {
15
16     @Query("SELECT u FROM Usuario u WHERE u.nombre = :nombre")
17     public Usuario buscarPorNombre(@Param("nombre") String nombre);
18 }
```

LIVE CODING

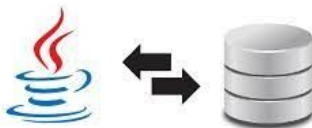
Completando la Wallet:

- Vamos a crear las interfaces **repositorio** para Usuario y Cuenta
- Además, vamos a definir los métodos de búsqueda por mail para usuario.

Tiempo: 15 minutos

➤ Asociaciones

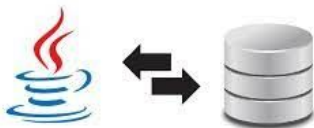
Asociaciones



JAVA PERSISTENCE API

Las asociaciones entre entidades son un aspecto clave en el modelado de bases de datos relacionales mediante Java Persistence API (JPA). Estas asociaciones definen cómo se relacionan las entidades entre sí, permitiendo representar conexiones lógicas y estructuras más complejas en una base de datos

Asociaciones



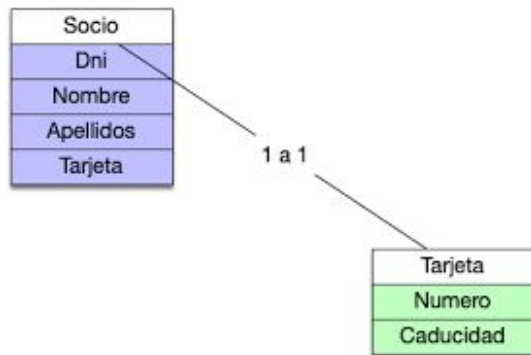
JAVA PERSISTENCE API

JPA nos brinda anotaciones para definir una relación entre dos clases que deseamos reflejar en la base de datos. Éstas anotaciones sirven para informarle a la base de datos cuál será el tipo de relación que tendrán las tablas entre sí. Recordemos que las anotaciones cumplen el propósito de “traducir” nuestro código de Java para que lo entienda la base de datos.

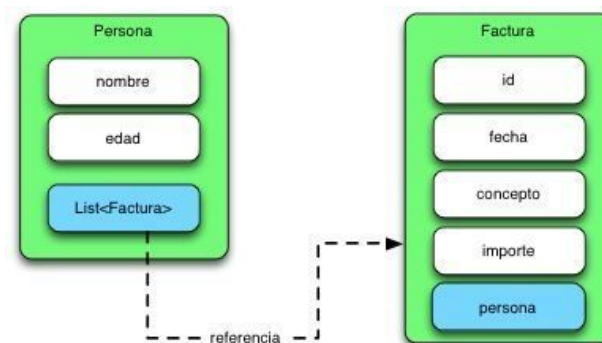
Asociaciones

Las anotaciones de asociación que nos provee JPA son:

@OneToOne : relación entre tablas uno a uno muchos.



@OneToMany : relación entre tablas uno a muchos.



Asociación Uno a Uno

La relación **One-to-One** (uno a uno) en JPA establece una asociación entre dos entidades donde una instancia de una entidad se relaciona con exactamente una instancia de otra entidad. En la entidad principal, se utiliza la anotación `@OneToOne` junto con `@JoinColumn` para especificar la columna que almacena la clave externa.

```
9 @Entity
10 public class Usuario {
11
12     @Id
13     @GeneratedValue(generator = "uuid")
14     private String id;
15     private String nombre;
16     private String email;
17
18     [
19     @OneToOne
20     @JoinColumn(name = "cuenta_id")
21     private Cuenta cuenta;
22     ]
23 }
```

Asociación Uno a Uno

En la entidad asociada, se utiliza `@OneToOne(mappedBy = "nombrePropiedad")` para establecer la relación **bidireccional**. Esta relación permite acceder a la entidad asociada desde la entidad principal y viceversa.

```
@Entity
public class Cuenta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "usuario")
    private Usuario usuario;
}
```

Asociación Uno a Muchos

La relación **One-to-Many** en JPA establece una asociación **entre dos entidades donde una entidad principal tiene una colección de entidades asociadas**. En la entidad principal, se utiliza la anotación **@OneToMany** para definir la relación, y en la entidad asociada no es necesario determinar ninguna anotación para establecer la relación con la entidad principal.

Asociación Uno a Muchos

De esta manera, se crea una relación **One-to-Many** donde un Usuario puede tener múltiples Cuentas.

Por otro lado, la entidad Cuenta **no tiene una referencia directa** a la entidad Usuario. La relación se establece a través de la columna cuenta_id en la tabla Usuario.

```
0 @Entity
1 public class Usuario {
2
3     @Id
4     @GeneratedValue(generator = "uuid")
5     private String id;
6     private String nombre;
7     private String email;
8
9     @OneToMany
10    private List<Cuenta> listaCuentas;
11
12 }
```

Asociaciones

Consideraciones importantes :

Bidireccional vs. Unidireccional : Las asociaciones pueden ser bidireccionales (ambas entidades tienen referencias entre sí) o unidireccionales (una entidad tiene una referencia a la otra). La elección depende de los requisitos específicos de la aplicación.

Optimización de Consultas : Al trabajar con asociaciones, es importante considerar la optimización de consultas, ya que las consultas relacionadas pueden afectar significativamente el rendimiento. El uso adecuado de **FetchType** y la comprensión de la carga diferida pueden ayudar a evitar problemas de rendimiento.

Asociaciones

Consideraciones importantes :

Fetch Type: El atributo **fetch** en las anotaciones **@OneToOne** y **@OneToMany** define cómo se deben cargar los datos asociados. El valor predeterminado es **FetchType.LAZY**, lo que significa que la carga se realiza de forma diferida. Si se utiliza **FetchType.EAGER**, los datos asociados se cargarán de inmediato.

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue(generator = "uuid")
    private String id;
    private String nombre;
    private String email;

    @OneToOne(fetch = FetchType.LAZY)
    private Cuenta cuenta;
}
```

LIVE CODING

Completando la Wallet:

Es momento de vincular nuestras entidades para **relacionar a cada usuario con su cuenta bancaria**.

Para esto vamos a realizar algunos cambios:

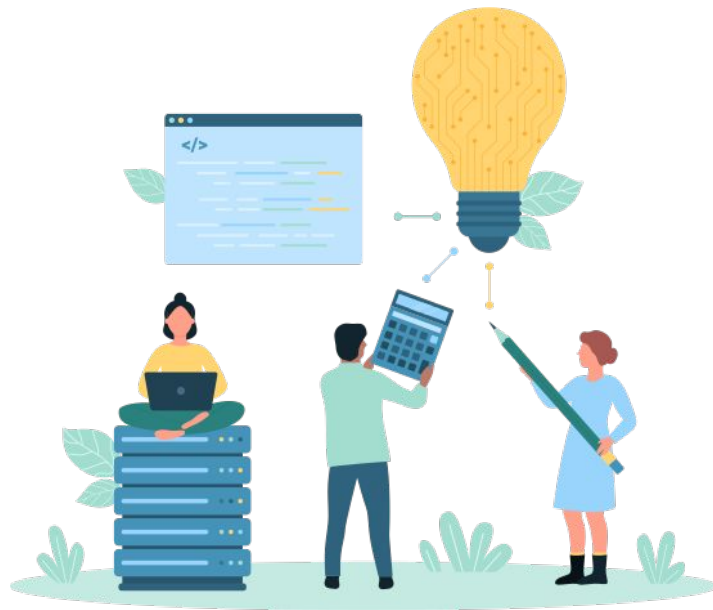
- **Eliminar** las clases “DAO” creadas con la implementación de JDBCTemplate (ya que utilizaremos los repositorios JPA).
- En la entidad Usuario, agregar el atributo Cuenta, con una relación uno-a-uno y un atributo de carga EAGER.

Tiempo: 15 minutos

➤ Invocar un repositorio desde un servicio

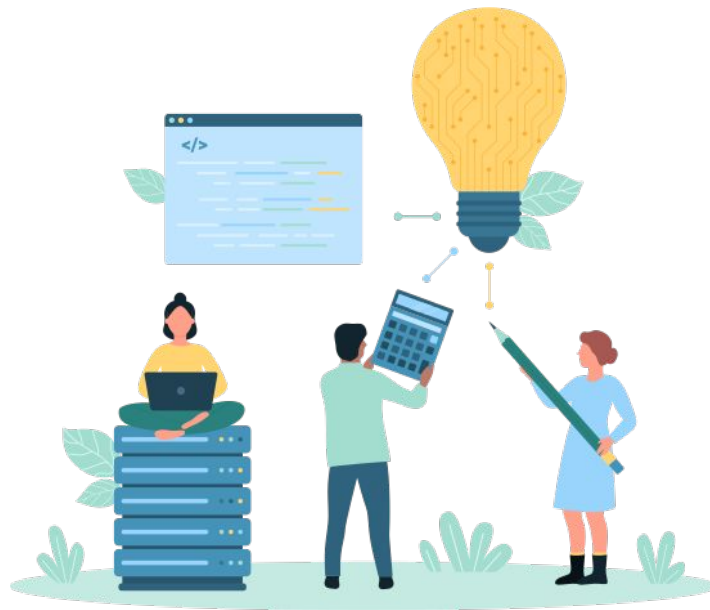
Invocar un repositorio desde un servicio

Invocar un repositorio desde un servicio es una práctica común en el desarrollo de aplicaciones Spring con JPA. Los servicios actúan como una capa intermedia entre el controlador (o cualquier otra capa de presentación) y el repositorio, encapsulando la lógica de negocio y permitiendo la interacción con la capa de acceso a datos.



Invocar un repositorio desde un servicio

Básicamente al invocar el repositorio desde el servicio puedes acceder a todas las operaciones definidas en el repositorio, como consultas personalizadas, operaciones de persistencia, eliminación, etc. Esto te permite encapsular la lógica de acceso a datos en el repositorio y utilizarla de manera conveniente desde el servicio.



Invocar un repositorio desde un servicio

¿Cómo invocar un repositorio desde el servicio?

1- **Inyección de Dependencia** : La anotación **@Autowired** se utiliza para inyectar la implementación de repositorio en el servicio. Esta es una práctica común en Spring para lograr la inversión de control y la inyección de dependencias. Veamos un ejemplo:

```
7
8 @Service
9 public class UsuarioServicio {
10
11     @Autowired
12     private final UsuarioRepositorio usuarioRepositorio;
13
14
15     public UsuarioServicio(UsuarioRepositorio usuarioRepositorio) {
16         this.usuarioRepositorio = usuarioRepositorio;
17     }
18 }
```

Invocar un repositorio desde un servicio

2- Operaciones con el repositorio: Es hora de crear los métodos encargados de invocar al repositorio para que realice los cambios persistentes en la base de datos. Por ejemplo:

obtenerUsuarioPorId(Long id) : Utiliza el método findById del repositorio para recuperar un usuario por su identificador.

obtenerUsuarioPorNombre (String nombre) : Utiliza un método personalizado del repositorio (buscarPorNombre) para recuperar un usuario por su nombre.

```
@Service
public class UsuarioServicio {

    @Autowired
    private UsuarioRepositorio usuarioRepositorio;

    public Usuario obtenerUsuarioPorId(Long id) {
        return usuarioRepositorio.findById(id).orElse(null);
    }

    public Usuario obtenerUsuariosPorNombre(String nombre) {
        return usuarioRepositorio.buscarPorNombre(nombre);
    }

    public void guardarUsuario(Usuario usuario) {
        usuarioRepositorio.save(usuario);
    }

    public void eliminarUsuario(Long id) {
        usuarioRepositorio.deleteById(id);
    }

    public List<Usuario> obtenerUsuarios() {
        return usuarioRepositorio.findAll();
    }
}
```

Invocar un repositorio desde un servicio

guardarUsuario(Usuario usuario) : Utiliza el método save del repositorio para guardar un usuario en la base de datos. La implementación de save determinará si se trata de una operación de creación o actualización.

eliminarUsuario(Long id): Utiliza el método deleteById del repositorio para eliminar un usuario por su identificador.

obtenerUsuarios() : Utiliza el método findAll del repositorio para recuperar una lista de usuarios.

```
@Service
public class UsuarioServicio {

    @Autowired
    private UsuarioRepositorio usuarioRepositorio;

    public Usuario obtenerUsuarioPorId(Long id) {
        return usuarioRepositorio.findById(id).orElse(null);
    }

    public Usuario obtenerUsuariosPorNombre(String nombre) {
        return usuarioRepositorio.buscarPorNombre(nombre);
    }

    public void guardarUsuario(Usuario usuario) {
        usuarioRepositorio.save(usuario);
    }

    public void eliminarUsuario(Long id) {
        usuarioRepositorio.deleteById(id);
    }

    public List<Usuario> obtenerUsuarios() {
        return usuarioRepositorio.findAll();
    }
}
```


Invocar un repositorio desde un servicio

Lógica de Negocio Adicional:

El servicio puede contener lógica de negocio adicional según los requisitos de la aplicación. Por ejemplo, se podrían agregar métodos que realizan operaciones más complejas que involucran la manipulación de varios objetos Usuario o la aplicación de reglas de negocio específicas.

```
@Service
public class UsuarioServicio {

    @Autowired
    private UsuarioRepositorio usuarioRepositorio;

    public Usuario obtenerUsuarioPorId(Long id) {
        return usuarioRepositorio.findById(id).orElse(null);
    }

    public Usuario obtenerUsuariosPorNombre(String nombre) {
        return usuarioRepositorio.buscarPorNombre(nombre);
    }

    public void guardarUsuario(Usuario usuario) {
        usuarioRepositorio.save(usuario);
    }

    public void eliminarUsuario(Long id) {
        usuarioRepositorio.deleteById(id);
    }

    public List<Usuario> obtenerUsuarios() {
        return usuarioRepositorio.findAll();
    }
}
```

Invocar un repositorio desde un servicio

Lógica de Negocio Adicional:

Por ejemplo este método **guardarUsuario** que recibe como parámetro el nombre y el email. El método se encarga de realizar la lógica correspondiente y luego se llama al repositorio para que persista los cambios en la base de datos.

```
@Autowired
private UsuarioRepositorio usuarioRepositorio;

public void guardarUsuario(String nombre, String email) {
    Usuario usuario = new Usuario();

    usuario.setNombre(nombre);
    usuario.setEmail(email);

    usuarioRepositorio.save(usuario);
}
```

LIVE CODING

Completando la Wallet: Es momento de operar en la base de datos con JPA.

- *Vamos a crear los métodos CRUD en los servicios de Usuario y de Cuenta y a invocar al repositorio de cada entidad para poder persistir los cambios en una base de datos.*

Tiempo: 30 minutos

➤ Transaccionalidad

Transaccionalidad

La transaccionalidad es un concepto fundamental en el desarrollo de aplicaciones que involucran operaciones sobre bases de datos. Proporciona un mecanismo para garantizar la integridad y la consistencia de los datos en situaciones donde varias operaciones deben ejecutarse como una unidad atómica. En el contexto de servicios y bases de datos, la transaccionalidad juega un papel crucial para **mantener la coherencia de los datos y garantizar la ejecución exitosa o fallida de un conjunto de operaciones relacionadas** .

Transaccionalidad

Una transacción de base de datos es un conjunto de instrucciones que se ejecutan en bloque . Por ejemplo, decidimos modificar un registro “A” en la base de datos. Si en alguna de las instrucciones se produce un error **todo el proceso se echa atrás** . De esta manera, si luego consultamos la base de datos podremos ver que el registro “A” no ha sido alterado.

A este proceso de “**tirar atrás**” las instrucciones realizadas se le dice hacer un **rollback** , mientras que el proceso de **confirmar** todas las instrucciones en bloque una vez hemos visto que no se ha producido ningún error se le llama hacer un **commit** .

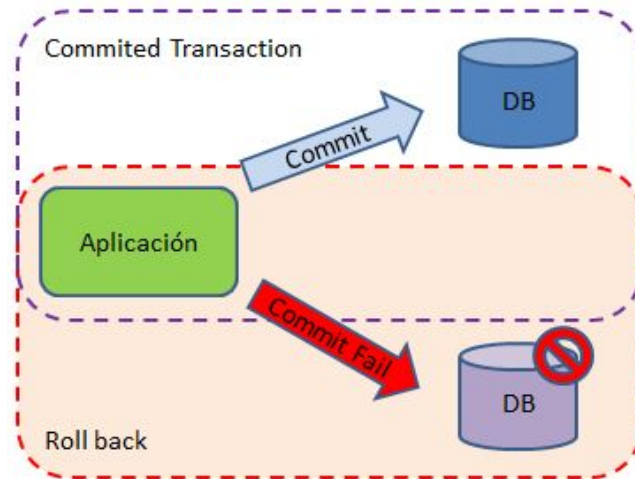


Transaccionalidad

Las transacciones cumplen con el concepto **ACID**:

Atomicidad : Garantiza que todas las operaciones de una transacción se completen con éxito o ninguna se complete en absoluto. Si alguna operación falla, la transacción se revierte a su estado original (rollback).

Consistencia : Asegura que la base de datos se mueva de un estado consistente a otro estado consistente. Cada transacción debe llevar la base de datos de un estado válido a otro.

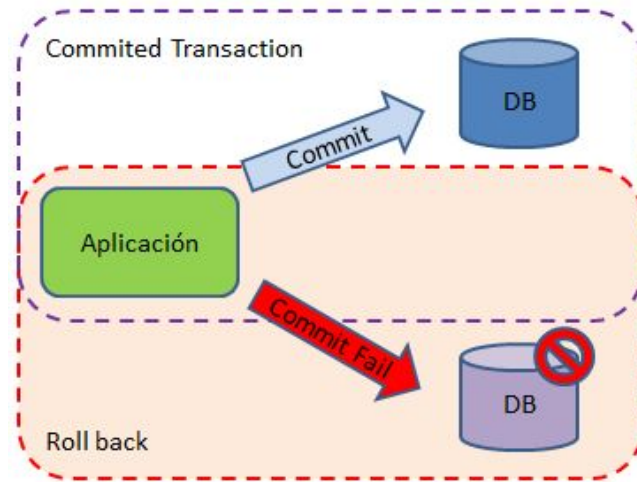


Transaccionalidad

Las transacciones cumplen con el concepto **ACID**:

Aislamiento : Garantiza que el efecto de una transacción no sea visible para otras transacciones hasta que se complete. Las transacciones deben ejecutarse de manera aislada entre sí.

Durabilidad : Una vez que una transacción se completa con éxito, sus cambios se mantienen incluso en caso de falla del sistema o reinicio. La durabilidad garantiza que los cambios sean permanentes.



➤ Configuración de la transaccionalidad

Configuración de la transaccionalidad

La manera más común de tratar las transacciones en Spring es mediante la anotación **@Transactional** en la cabecera del método de una clase (nunca una interfaz) gestionada por Spring.

En nuestro caso, los métodos encargados de gestionar transacciones se encontrarán en las clases de servicio o lógica de negocio de nuestra aplicación.

```
@Transactional
public void hazAlgoTransaccionalmente() {
    // Soy transaccional!
}
```

Configuración de la transaccionalidad

- **Iniciación Automática de Transacciones:** Si un método anotado con `@Transactional` se llama desde otro método, una nueva transacción se inicia si aún no existe. Si la transacción actual tiene éxito, se confirma al final del método; de lo contrario, se revierte.
- **Rollback Automático:** Si una excepción no controlada se lanza durante la ejecución de un método anotado con `@Transactional`, Spring revierte automáticamente la transacción, deshaciendo cualquier cambio realizado durante la transacción.
- **Configuración Personalizada:** Spring permite configuración personalizada, como definir la propagación de transacciones, el aislamiento y otros aspectos a través de atributos en la anotación `@Transactional`.

Configuración de la transaccionalidad

Propagación de Transacciones en Spring:

La propagación de transacciones en Spring se refiere a cómo se manejan las transacciones cuando un método anotado con `@Transactional` llama a otro método que posee la misma anotación. Algunos de los modos de propagación más comunes son:

- **REQUIRED** (Predeterminado): Si ya existe una transacción, el método se une a ella; de lo contrario, se inicia una nueva transacción. Es el comportamiento predeterminado y el más utilizado.
- **REQUIRES_NEW** : Si ya existe una transacción, se suspende y se inicia una nueva transacción. Útil cuando deseas que el método tenga su propia transacción independientemente de la existencia de una transacción anterior.



Configuración de la transaccionalidad

Propagación de Transacciones en Spring:

- **NESTED** : Si existe una transacción, crea un nuevo punto de guardado dentro de esa transacción. Si no hay transacción, se comporta como REQUIRED.
- **PROPAGATION_SUPPORTS** : Si existe transacción la aprovecha, sino no crea ninguna.
- **PROPAGATION_MANDATORY** : Si no existe una transacción abierta se lanza una excepción. Hay programadores que anotan sus DAO con esta opción.
- **PROPAGATION_NOT_SUPPORTED** : Si existe una transacción la pone en suspenso, la transacción se reactiva al salir del método.

Configuración de la transaccionalidad

Propagación de Transacciones en Spring:

La configuración de la propagación de las transacciones se realizan mediante la escritura de estas órdenes en el atributo “propagation” de la anotación @Transactional.

```
@Transactional(propagation = Propagation.MANDATORY)
public void eliminarUsuario(Long id) {
    usuarioRepository.deleteById(id);
}
```

Configuración de la transaccionalidad

Aislamiento de Transacciones en Spring:

El aislamiento de transacciones en Spring controla cómo las transacciones concurrentes interactúan entre sí. Algunos niveles de aislamiento comunes son:

- **DEFAULT** (Predeterminado): Utiliza el nivel de aislamiento predeterminado de la base de datos.
- **READ_COMMITTED** : Evita la lectura de datos no comprometidos (datos modificados pero aún no confirmados) por otras transacciones.
- **READ_UNCOMMITTED** : Permite la lectura de datos no comprometidos por otras transacciones.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void metodoConTransaccionReadCommitted() {
    // Operaciones con la base de datos
}
```

Configuración de la transaccionalidad

También podemos configurar tanto la propagación como el aislamiento para el mismo método:

```
@Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.READ_COMMITTED)
public void metodoConTransaccionPersonalizada() {
    // Operaciones con la base de datos
}
```


Configuración de la transaccionalidad

Para poder manejar bien las transacciones, debemos tener en cuenta que **todo método que pueda generar un cambio persistente en la base de datos, debe ser marcado como @Transactional.**

En los casos de los **métodos de consulta** , podemos encontrarnos que también están anotados como transaccionales, pero en estos casos **debemos tener en cuenta su lectura** .

Configuración de la transaccionalidad

La autorización se configura con la propiedad `readOnly` (solo lectura) de `@Transactional`. Su valor predeterminado es `false`, lo que significa que no está en modo de solo lectura, ergo se permite la escritura. Dicho con otras palabras, `@Transactional` siempre otorga permiso de escritura a la transacción, salvo que se indique lo contrario de manera explícita. La forma de declararlo es la siguiente:

```
@Transactional(readOnly = true)
public Usuario obtenerUsuariosPorNombre(String nombre) {
    return usuarioRepositorio.buscarPorNombre(nombre);
}
```

➤ Creando un servicio transaccional



Creando un servicio transaccional

Crear un servicio transaccional en Spring implica utilizar la anotación `@Transactional` en los métodos del servicio que necesitan una gestión transaccional. En esta clase vamos a anotar los métodos que realizan transacciones en la base de datos.

Recordemos que en caso de que el método no vaya a realizar escrituras sobre la base de datos, podemos marcarlo como “solo lectura”, lo que le dará mayor beneficio de rendimiento. Cuando un método es anotado como `readOnly = true`, si se intenta realizar alguna operación de escritura sobre la BD, se generará una excepción.



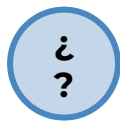
LIVE CODING

Transacciones en la AlkeWallet

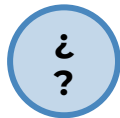
- Vamos a agregar la anotación `@Transactional` a todos los métodos de los servicios de Cuenta y Usuario.
- En caso de que el método sea de “sólo lectura” (no intenta persistir en la base de datos), deberemos anotararlo.
- Además, podemos probar las transacciones viendo el impacto en la base de datos.

Tiempo: 20 minutos

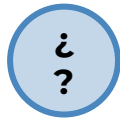
#Momentode Preguntas...



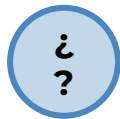
¿Qué ventajas aporta Spring Data JPA frente a JdbcTemplate?



¿Cuándo conviene usar relaciones bidireccionales?



¿Cómo afecta el uso de fetch = EAGER al rendimiento?



¿Cuál es la diferencia entre REQUIRES_NEW y REQUIRED en propagación?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Acceso a datos con JPA y repositorios

Acceso a datos con JPA y repositorios

Contexto: 🙌

Tu aplicación de billetera digital ya cuenta con la capa de persistencia usando JdbcTemplate, pero el equipo ha decidido migrar a Spring Data JPA para aprovechar la generación automática de consultas y simplificar el código.

Consigna: 🛠️

Reemplaza las clases DAO por interfaces repositorio (UsuarioRepositorio y CuentaRepositorio) usando CrudRepository. Agrega búsquedas personalizadas, relaciones entre entidades y gestiona transacciones de forma adecuada.

Tiempo 🕒: 30 minutos

Paso a paso: ⚙️

1. Elimina las clases DAO previas.
2. Crea las interfaces UsuarioRepositorio y CuentaRepositorio.
3. Declara métodos como findByEmail(String email) usando convención o @Query.
4. Enlaza Usuario con Cuenta mediante @OneToOne(fetch = FetchType.EAGER)
5. En los servicios, usa @Autowired para inyectar los repositorios.
6. Anota los métodos de escritura con @Transactional, y los de solo lectura con readOnly = true.
7. Probar funcionalidades desde el controlador.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Usamos JpaRepository para reemplazar DAOs
- ✓ Creamos consultas automáticas y personalizadas
- ✓ Relacionamos entidades con @OneToOne y @OneToMany
- ✓ Implementamos servicios con lógica de negocio intermedia
- ✓ Manejamos transacciones para mantener la integridad de datos
- ✓ Aprendimos a optimizar acceso con fetch, readOnly y niveles de aislamiento



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

